

SPI Driver Analysis – STM32F446RE

Introduction

The SPI driver is developed to simplify Serial Peripheral Interface (SPI) communication using the SPI peripherals available in the STM32F446RE microcontroller. Instead of directly accessing SPI registers in the application code, the driver provides a set of APIs that handle all required register-level operations internally. This approach improves code readability, reusability, and maintainability while reducing the complexity of peripheral configuration.

How the Driver Works

The SPI driver acts as an abstraction layer between the user application and the SPI hardware of the STM32F446RE microcontroller. When the application needs to perform SPI communication, it interacts with the SPI driver APIs rather than directly modifying hardware registers. During SPI initialization, the application first fills the SPI handle structure with the required communication parameters such as device mode, bus configuration, clock speed, data frame format, clock polarity, clock phase, and software slave management settings. The `SPI_Init()` API then reads these parameters and configures the corresponding SPI registers automatically.

This design allows different SPI peripherals to be configured using the same API without modifying the driver source code. As a result, the driver hides low-level register operations from the application and provides a simple and reusable interface for SPI communication.

For example:

- Device mode configuration → CR1 register
- Bus configuration → CR1 register
- Clock prescaler configuration → CR1 register
- Data frame format configuration → CR1 register
- Clock polarity configuration → CR1 register
- Clock phase configuration → CR1 register
- Software slave management configuration → CR1 register
- Data transmission and reception → DR register
- Status monitoring → SR register

The application does not need to know which register or bit position must be modified because the driver handles all register-level operations internally.

Hardware Abstraction

Without a driver, configuring an SPI peripheral requires direct register manipulation.

Example:

```
RCC->APB1ENR |= (1 << 14);
```

```
SPI2->CR1 |= (1 << 2);
```

```
SPI2->CR1 |= (1 << 6);
```

With the SPI driver:

```
SPI_Handle_t    SPI2Handle;
```

```
SPI2Handle.pSPIx = SPI2;
```

```
SPI2Handle.SPIConfig.SPI_DeviceMode = SPI_DEVICE_MODE_MASTER;
```

```
SPI2Handle.SPIConfig.SPI_BusConfig = SPI_BUS_CONFIG_FD;
```

```
SPI2Handle.SPIConfig.SPI_SclkSpeed = SPI_SCLK_SPEED_DIV8;
```

```
SPI2Handle.SPIConfig.SPI_DFF = SPI_DFF_8BITS;
```

```
SPI2Handle.SPIConfig.SPI_CPOL = SPI_CPOL_LOW;
```

```
SPI2Handle.SPIConfig.SPI_CPHA = SPI_CPHA_LOW;
```

```
SPI2Handle.SPIConfig.SPI_SSM = SPI_SSM_EN;
```

```
SPI_Init(&SPI2Handle);
```

The SPI driver automatically performs the required peripheral configuration based on the settings provided by the user. In this project, the driver:

- Configures the SPI peripheral for communication.
- Configures Master or Slave operation.
- Configures Full Duplex, Half Duplex, or Simplex communication.

- Sets the SPI clock speed.
- Configures clock polarity and clock phase.
- Configures data frame format.
- Configures software slave management.
- Updates the required SPI registers internally through the SPI_Init() API.

This approach hides low-level register manipulation from the application developer and provides a simple API-based interface for SPI configuration and communication.

SPI Driver APIs

The SPI_PeriClockControl() API is used to enable or disable the peripheral clock for an SPI module. Before configuring any SPI peripheral, the corresponding peripheral clock must be enabled. This API ensures that the SPI peripheral receives the required clock from the RCC module.

The SPI_Init() API is responsible for configuring an SPI peripheral according to the parameters specified in the SPI handle structure. It configures device mode, bus configuration, clock speed, data frame format, clock polarity, clock phase, and software slave management settings. This is the primary API used during SPI initialization.

The SPI_DeInit() API resets the selected SPI peripheral back to its default reset state. This can be useful when reconfiguring the peripheral or restoring default settings.

The SPI_SendData() API is used to transmit data using polling-based communication. The API waits until the transmit buffer becomes empty and then loads the next data byte into the SPI data register.

The SPI_ReceiveData() API is used to receive data using polling-based communication. The API continuously monitors the receive status flags and retrieves incoming data from the SPI peripheral.

The SPI_SendDataIT() API provides interrupt-based data transmission. Instead of continuously polling status flags, the driver uses interrupts to transmit data efficiently while allowing the CPU to perform other tasks.

The SPI_ReceiveDataIT() API provides interrupt-based data reception. Incoming data is received through interrupt events, improving system responsiveness and reducing CPU utilization.

The SPI_IRQInterruptConfig() API is used to enable or disable SPI interrupt requests within the NVIC controller.

The SPI_IRQPriorityConfig() API assigns interrupt priority levels to SPI interrupt sources, allowing proper interrupt management when multiple interrupts occur simultaneously.

The SPI_IRQHandling() API handles SPI interrupt events and processes transmission, reception, and error-related interrupt conditions.

The SPI_GetFlagStatus() API is used to check the status of SPI flags such as TXE, RXNE, and BSY.

The SPI_ClearOVRFlag() API clears the SPI overrun flag whenever required.

The SPI_PeripheralControl() API enables or disables the SPI peripheral after configuration.

The SPI_SSICongig() API configures the internal slave select signal when software slave management is enabled.

The SPI_SSOEConfig() API enables or disables automatic slave select output management.

Conclusion

The SPI driver developed for the STM32F446RE microcontroller provides a structured and reusable interface for SPI communication. The driver successfully abstracts low-level hardware operations and allows the application to interact with SPI peripherals through a simple set of APIs. By hiding register-level complexity and providing a modular software architecture, the SPI driver simplifies embedded application development, improves code maintainability, and enables reliable communication with SPI-based devices and debugging tools such as Logic Analyzer.